PHASE 5 OF 8

Local Model & Quiz Engine

How the on-device explainer works, and how the quiz generator turns model output into safe, structured JSON.

explanation_module.py | quiz_module.py | check_models.py

5.1 explanation_module.py — Concept Explainer

Purpose: generates a simple, student-friendly explanation of any topic using a small open-source model that runs on the local machine instead of calling an external API.

How it works

- Uses `MBZUAI/LaMini-Flan-T5-783M` via Hugging Face *transformers*.
- The tokenizer and model are loaded **once** and cached in memory (`_load_model()`), so repeated requests do not reload the model from disk.
- Wraps the topic in a prompt asking for a simple, clear explanation "for a school student."
- Generation runs inside `torch.no_grad()`, which is more memory-efficient because no training gradients are needed.

Simplified excerpt

```
_model_cache = {}

def _load_model():
    if "model" not in _model_cache:
        _model_cache["tok"] = AutoTokenizer.from_pretrained(MODEL_NAME)
        _model_cache["model"] = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
    return _model_cache["tok"], _model_cache["model"]

def explain_topic(topic: str) -> str:
    tok, model = _load_model()
    prompt = f"Explain {topic} simply for a school student."
    with torch.no_grad():
        ids = tok(prompt, return_tensors="pt").input_ids
        out = model.generate(ids, max_new_tokens=200)
    return tok.decode(out[0], skip_special_tokens=True)
```

Runs a local model — no internet needed once weights are cached.

Performance notes

Loading happens on the first request only; later requests reuse the in-memory model. Generation is capped at 200 new tokens, which keeps answers short and response times predictable on CPU-only machines.

5.2 quiz_module.py — Quiz Generator

Purpose: turns a passage into 3 multiple-choice questions, each with 4 options and one correct answer.

How it works

- Prompts Gemini to respond with **valid JSON only**, in a specific shape (question / options / answer).
- `clean_json_block()` strips Markdown code fences (```json … ```) in case the model wraps its answer in them anyway.
- Parses the cleaned text with `json.loads()` and checks the result is a list, so a malformed response returns a friendly error instead of crashing the app.

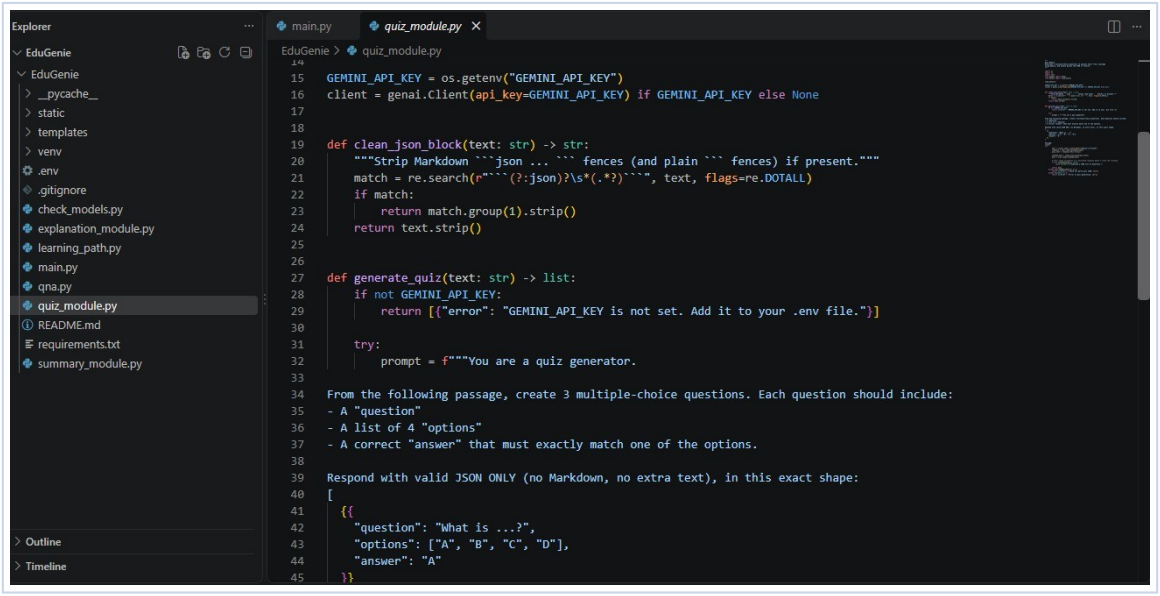


Figure — quiz_module.py: prompt construction and the JSON-fence-stripping helper.

Expected JSON shape

```
[
  {
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }
]
```

Each *answer* must exactly match one of the *options*, which is what allows the browser to grade each question instantly without another server call.

Prompt design tips for reliable JSON

- State the exact shape and demand “valid JSON only” with no Markdown or commentary.
- Fix the number of questions and options so the UI can render a predictable layout.
- Require each answer to match an option exactly — the browser compares strings to grade.

5.3 Quiz Pipeline

Stage	What happens	Safeguard
1. Key check	Confirms GEMINI_API_KEY is available	Returns an error object if missing
2. Prompt	Asks for exactly 3 questions as strict JSON	No Markdown, no extra text requested
3. Clean	`clean_json_block()` removes code fences	Handles ```json and plain ``` fences
4. Parse	`json.loads()` converts text to Python	Wrapped in error handling
5. Validate	Confirms the result is a list	Wrong shape returns a friendly error
6. Respond	Route returns {“quiz”: [...]}	Frontend renders auto-graded radio buttons

5.4 Quiz Failure Modes

Failure	Cause	Mitigation
Parsing error	Model adds commentary or a badly formed response	Rerun the request; tighten the prompt’s formatting rules
Wrong JSON shape	Model returns an object instead of a list	Shape validation returns a friendly error
Fenced output	Model wraps JSON in Markdown fences	`clean_json_block()` strips them
Missing key	.env not loaded	Error object is returned instead of a crash

5.5 check_models.py — Utility Script

Purpose: a standalone helper (not part of the running app) that shows which Gemini models your API key can use.

- Lists every model available to your key and prints the ones that support `generateContent` — the operation used for chat and text generation.
- Useful for confirming a model name such as `gemini-3.6-flash` is valid before debugging “model not found” errors elsewhere.

> **Note on the quiz input**
> The prompt is written for a *passage*, but the UI example (“python”) shows a short *topic* also works. For richer, more accurate questions, paste a few sentences of source material.